aioauth

Release 1.5.3

Ali Aliyev

May 06, 2023

CONTENTS

1 Installing **3**

2 Supported RFC **5**

3 Pages **7**

4 Sections **9**

4.1 Plug-and-Play . . . 9
4.2 Configuration . . . 9
4.3 Server & Database . . . 10
4.4 Aiohttp . . . 10
4.5 FastAPI . . . 10
4.6 Collections . . . 11
4.7 Config . . . 11
4.8 Constances . . . 12
4.9 Errors . . . 12
4.10 Grant Type . . . 14
4.11 Models . . . 15
4.12 Requests . . . 18
4.13 Response Type . . . 19
4.14 Responses . . . 20
4.15 Server . . . 21
4.16 Storage . . . 24
4.17 Types . . . 27
4.18 Utils . . . 28

5 Indices and tables **31**

Python Module Index **33**

Index **35**

aioauth is a spec-compliant OAuth 2.0 asynchronous Python module. aioauth works out-of-the-box with asynchronous server frameworks like FastAPI, Starlette, aiohttp, and others, as well as asynchronous database modules like Motor (MongoDB), aiopg (PostgreSQL), aiomysql (MySQL), or ORMs like Gino, sqlalchemy, or Tortoise.

The magic of aioauth is its plug-and-play methods that allow the use of virtually any server or database framework.

CHAPTER

ONE

INSTALLING

To install aioauth at the command line:

```
$ pip install aioauth
```

To install pre-releases:

```
$ pip install git+https://github.com/aliev/aioauth
```

CHAPTER

TWO

SUPPORTED RFC

aioauth supports the following RFCs:

- RFC 6749 - The OAuth 2.0 Authorization Framework
- RFC 7662 - OAuth 2.0 Token Introspection
- RFC 7636 - Proof Key for Code Exchange by OAuth Public Clients

CHAPTER THREE

PAGES

- Github Project
- Issues
- Discussion

CHAPTER

FOUR

SECTIONS

4.1 Plug-and-Play

aioauth was designed to be as flexible as possible to allow developers to choose their own server framework, as well as database provider. Since aioauth is written as an asynchronous module it would be to the advantage of the developer to write their applications asynchronously.

4.2 Configuration

All aioauth settings are made through `aioauth.config.Settings` class.

Defaults

Setting	Default value	Description
TOKEN_EXPIRES_IN	86400	Access token lifetime.
AUTHORIZATION_CODE_EXPIRES_IN	300	Authorization code lifetime.
INSECURE_TRANSPORT	False	Allow connections over SSL only. When this option is disabled server will raise "HTTP method is not allowed" error.

the default settings can be changed as follows:

```
import os
from aioauth.config import Settings

settings = Settings(
    INSECURE_TRANSPORT=not os.getenv('DEBUG', False)
)
```

this example disables checking for insecure transport, depending on the debug mode of the current environment.

The `aioauth.requests.Request` consumes an instance of the `aioauth.config.Settings` class:

```
import os
from aioauth.config import Settings
from aioauth.requests import Request

settings = Settings(
    INSECURE_TRANSPORT=not os.getenv('DEBUG', False)
```

(continues on next page)

(continued from previous page)

```
)

request = Request(
    settings=settings,
    ...
)
```

4.3 Server & Database

4.4 Aiohttp

4.5 FastAPI

4.5.1 Installing

To install aioauth with FastAPI at the command line:

```
$ pip install aioauth[fastapi]
```

Usage example

```
from dataclasses import dataclasses
from aioauth_fastapi.router import get_oauth2_router
from aioauth.storage import BaseStorage
from aioauth.requests import BaseRequest, Query, Post
from aioauth.models import AuthorizationCode, Client, Token
from aioauth.config import Settings
from aioauth.server import AuthorizationServer
from fastapi import FastAPI

app = FastAPI()

@dataclasses
class User:
    """Custom user model"""
    first_name: str
    last_name: str


class Request(BaseRequest[Query, Post, User]):
    """Custom Request model"""


class Storage(BaseStorage[Token, Client, AuthorizationCode, Request]):
    """
    Storage methods must be implemented here.
    """

storage = Storage()
authorization_server = AuthorizationServer[Request, Storage](storage)
```

(continues on next page)

(continued from previous page)

```
# NOTE: Redefinition of the default aioauth settings
# INSECURE_TRANSPORT must be enabled for local development only!
settings = Settings(
    INSECURE_TRANSPORT=True,
)

# Include FastAPI router with oauth2 endpoints.
app.include_router(
    get_oauth2_router(authorization_server, settings),
    prefix="/oauth2",
    tags=["oauth2"],
)
```

4.6 Collections

```
from aioauth import collections
```

Collections that are used throughout the project.

---

class HTTPHeaderDict(*dict=None*, */*, ***kwargs*)

Parameters

- **headers** – An iterable of field-value pairs. Must not contain multiple field names when compared case-insensitively.
- **kwargs** – Additional field-value pairs to pass in to `dict.update`.

A `dict` like container for storing HTTP Headers.

Example:

```
from aioauth.collections import HTTPHeaderDict
d = HTTPHeaderDict({"hello": "world"})
d['hello'] == 'world' # >>> True
d['Hello'] == 'world' # >>> True
d['hElLo'] == 'world' # >>> True
```

4.7 Config

```
from aioauth import config
```

Configuration settings for aioauth server instance.

---

class Settings(*TOKEN_EXPIRES_IN: int = 86400*, *REFRESH_TOKEN_EXPIRES_IN: int = 172800*, *AUTHORIZATION_CODE_EXPIRES_IN: int = 300*, *INSECURE_TRANSPORT: bool = False*, *ERROR_URI: str = ''*, *AVAILABLE: bool = True*)

Configuration options that is used by the Server class.

TOKEN_EXPIRES_IN: int = 86400

Access token lifetime in seconds. Defaults to 24 hours.

REFRESH_TOKEN_EXPIRES_IN: int = 172800
Refresh token lifetime in seconds. Defaults to TOKEN_EXPIRES_IN * 2 (48 hours).

AUTHORIZATION_CODE_EXPIRES_IN: int = 300
Authorization code lifetime in seconds. Defaults to 5 minutes.

INSECURE_TRANSPORT: bool = False
Allow connections over SSL only.

> **Note:** When this option is disabled server will raise "HTTP method is not allowed" error when attempting to access the server without a valid SSL tunnel.

ERROR_URI: str = ''
URI to redirect resource owner when server encounters error.

AVAILABLE: bool = True
Boolean indicating whether or not the server is available.

4.8 Constances

```
from aioauth import constances
```

Constants that are used throughout the project.

---

default_headers = {'content-type': 'application/json', 'cache-control': 'no-store', 'prag
The authorization server **must** include the HTTP `Cache-Control` response header field, as per RFC2616, with a value of `no-store` in any response containing tokens, credentials, or other sensitive information, as well as the `Pragma` response header field, as per RFC2616, with a value of `no-cache`.

4.9 Errors

```
from aioauth import errors
```

Errors used throughout the project.

---

exception MethodNotAllowedError(*request: TRequest, description: Optional[str] = None, headers: Optional[*aioauth.collections.HTTPHeaderDict*] = None*)
The request is valid, but the method trying to be accessed is not available to the resource owner.

description: str = 'HTTP method is not allowed.'

status_code: http.HTTPStatus = 405

error: Literal[invalid_request, invalid_client, invalid_grant, invalid_scope, unautho

exception InvalidRequestError(*request: TRequest, description: Optional[str] = None, headers: Optional[*aioauth.collections.HTTPHeaderDict*] = None*)
The request is missing a required parameter, includes an invalid parameter value, includes a parameter more than once, or is otherwise malformed.

error: Literal[invalid_request] = 'invalid_request'

exception InvalidClientError(*request: TRequest, description: Optional[str] = None, headers: Optional[aioauth.collections.HTTPHeaderDict] = None*)

Client authentication failed (e.g. unknown client, no client authentication included, or unsupported authentication method). The authorization server **may** return an `HTTP 401` (Unauthorized) status code to indicate which HTTP authentication schemes are supported. If the client attempted to authenticate via the `Authorization` request header field, the authorization server **must** respond with an `HTTP 401` (Unauthorized) status code, and include the `WWW-Authenticate` response header field matching the authentication scheme used by the client.

error: Literal[invalid_request, invalid_client, invalid_grant, invalid_scope, unautho

status_code: http.HTTPStatus = 401

exception InsecureTransportError(*request: TRequest, description: Optional[str] = None, headers: Optional[aioauth.collections.HTTPHeaderDict] = None*)

An exception will be thrown if the current request is not secure.

description: str = 'OAuth 2 MUST utilize https.'

error: Literal[invalid_request, invalid_client, invalid_grant, invalid_scope, unautho

exception UnsupportedGrantTypeError(*request: TRequest, description: Optional[str] = None, headers: Optional[aioauth.collections.HTTPHeaderDict] = None*)

The authorization grant type is not supported by the authorization server.

error: Literal[invalid_request, invalid_client, invalid_grant, invalid_scope, unautho

exception UnsupportedResponseTypeError(*request: TRequest, description: Optional[str] = None, headers: Optional[aioauth.collections.HTTPHeaderDict] = None*)

The authorization server does not support obtaining an authorization code using this method.

error: Literal[invalid_request, invalid_client, invalid_grant, invalid_scope, unautho

exception InvalidGrantError(*request: TRequest, description: Optional[str] = None, headers: Optional[aioauth.collections.HTTPHeaderDict] = None*)

The provided authorization grant (e.g. authorization code, resource owner credentials) or refresh token is invalid, expired, revoked, does not match the redirection URI used in the authorization request, or was issued to another client.

See RFC6749 section 5.2.

error: Literal[invalid_request, invalid_client, invalid_grant, invalid_scope, unautho

exception MismatchingStateError(*request: TRequest, description: Optional[str] = None, headers: Optional[aioauth.collections.HTTPHeaderDict] = None*)

Unable to securely verify the integrity of the request and response.

description: str = 'CSRF Warning! State not equal in request and response.'

error: Literal[mismatching_state] = 'mismatching_state'

exception UnauthorizedClientError(*request: TRequest, description: Optional[str] = None, headers: Optional[aioauth.collections.HTTPHeaderDict] = None*)

The authenticated client is not authorized to use this authorization grant type.

error: Literal[invalid_request, invalid_client, invalid_grant, invalid_scope, unautho

exception InvalidScopeError(*request: TRequest, description: Optional[str] = None, headers: Optional[aioauth.collections.HTTPHeaderDict] = None*)

The requested scope is invalid, unknown, or malformed, or exceeds the scope granted by the resource owner.

See RFC6749 section 5.2.

error: Literal[invalid_request, invalid_client, invalid_grant, invalid_scope, unautho

exception ServerError(*request: TRequest, description: Optional[str] = None, headers: Optional[aioauth.collections.HTTPHeaderDict] = None*)

The authorization server encountered an unexpected condition that prevented it from fulfilling the request. (This error code is needed because a `HTTP 500` (Internal Server Error) status code cannot be returned to the client via a HTTP redirect.)

error: Literal[invalid_request, invalid_client, invalid_grant, invalid_scope, unautho

exception TemporarilyUnavailableError(*request: TRequest, description: Optional[str] = None, headers: Optional[aioauth.collections.HTTPHeaderDict] = None*)

The authorization server is currently unable to handle the request due to a temporary overloading or maintenance of the server. (This error code is needed because a `HTTP 503` (Service Unavailable) status code cannot be returned to the client via a HTTP redirect.)

error: Literal[invalid_request, invalid_client, invalid_grant, invalid_scope, unautho

exception InvalidRedirectURIError(*request: TRequest, description: Optional[str] = None, headers: Optional[aioauth.collections.HTTPHeaderDict] = None*)

The requested redirect URI is missing or not allowed.

error: Literal[invalid_request, invalid_client, invalid_grant, invalid_scope, unautho

exception UnsupportedTokenTypeError(*request: TRequest, description: Optional[str] = None, headers: Optional[aioauth.collections.HTTPHeaderDict] = None*)

The authorization server does not support the revocation of the presented token type. That is, the client tried to revoke an access token on a server not supporting this feature.

error: Literal[invalid_request, invalid_client, invalid_grant, invalid_scope, unautho

4.10 Grant Type

```
from aioauth import grant_type
```

Different OAuth 2.0 grant types.

---

class GrantTypeBase(*storage: TStorage, client_id: str, client_secret: Optional[str]*)

Base grant type that all other grant types inherit from.

async create_token_response(*request: TRequest, client: aioauth.models.Client*) → *aioauth.responses.TokenResponse*

Creates token response to reply to client.

async validate_request(*request: TRequest*) → *aioauth.models.Client*

Validates the client request to ensure it is valid.

class AuthorizationCodeGrantType(*storage: TStorage, client_id: str, client_secret: Optional[str]*)

The Authorization Code grant type is used by confidential and public clients to exchange an authorization code for an access token. After the user returns to the client via the redirect URL, the application will get the authorization code from the URL and use it to request an access token. It is recommended that all clients use RFC 7636 Proof Key for Code Exchange extension with this flow as well to provide better security.

Note: Note that `aioauth` implements RFC 7636 out-of-the-box. See RFC 6749 section 1.3.1.

async validate_request(*request: TRequest*) → *aioauth.models.Client*
Validates the client request to ensure it is valid.

async create_token_response(*request: TRequest*, *client:* aioauth.models.Client) → *aioauth.responses.TokenResponse*
Creates token response to reply to client.

class PasswordGrantType(*storage: TStorage*, *client_id: str*, *client_secret: Optional[str]*)
The Password grant type is a way to exchange a user's credentials for an access token. Because the client application has to collect the user's password and send it to the authorization server, it is not recommended that this grant be used at all anymore. See RFC 6749 section 1.3.3. The latest OAuth 2.0 Security Best Current Practice disallows the password grant entirely.

async validate_request(*request: TRequest*) → *aioauth.models.Client*
Validates the client request to ensure it is valid.

class RefreshTokenGrantType(*storage: TStorage*, *client_id: str*, *client_secret: Optional[str]*)
The Refresh Token grant type is used by clients to exchange a refresh token for an access token when the access token has expired. This allows clients to continue to have a valid access token without further interaction with the user. See RFC 6749 section 1.5.

async create_token_response(*request: TRequest*, *client:* aioauth.models.Client) → *aioauth.responses.TokenResponse*
Validate token request and create token response.

async validate_request(*request: TRequest*) → *aioauth.models.Client*
Validates the client request to ensure it is valid.

class ClientCredentialsGrantType(*storage: TStorage*, *client_id: str*, *client_secret: Optional[str]*)
The Client Credentials grant type is used by clients to obtain an access token outside of the context of a user. This is typically used by clients to access resources about themselves rather than to access a user's resources. See RFC 6749 section 4.4.

async validate_request(*request: TRequest*) → *aioauth.models.Client*
Validates the client request to ensure it is valid.

4.11 Models

```
from aioauth import models
```

Memory objects used throughout the project.

class Client(*client_id: str*, *client_secret: str*, *grant_types: List[Literal[authorization_code, password, client_credentials, refresh_token]]*, *response_types: List[Literal[token, code, none, id_token]]*, *redirect_uris: List[str]*, *scope: str = ''*, *user: Optional[Any] = None*)
OAuth2.0 client model object.

client_id: str
Public identifier for the client. It must also be unique across all clients that the authorization server handles.

client_secret: str
Client secret is a secret known only to the client and the authorization server. Used for secure communication between the client and authorization server.

grant_types: List[Literal[authorization_code, password, client_credentials, refresh_t
The method(s) in which an application gets an access token from the provider. Each grant type is optimized for a particular use case, whether that's a web app, a native app, a device without the ability to launch a web browser, or server-to-server applications.

response_types: List[Literal[token, code, none, id_token]]
A list containing the types of the response expected.

redirect_uris: List[str]
After a user successfully authorizes an application, the authorization server will redirect the user back to the application with either an authorization code or access token in the URL. Because the redirect URL will contain sensitive information, it is critical that the service doesn't redirect the user to arbitrary locations.

scope: str = ''
Scope is a mechanism that limit an application's access to a user's account. An application can request one or more scopes, this information is then presented to the user in the consent screen, and the access token issued to the application will be limited to the scopes granted.

user: Optional[Any] = None
The user who is the creator of the Client. This optional attribute can be useful if you are creating a server that can be managed by multiple users.

check_redirect_uri(*redirect_uri*) → bool
Verifies passed `redirect_uri` is part of the Clients's `redirect_uris` list.

check_grant_type(*grant_type: Optional[Literal[authorization_code, password, client_credentials, refresh_token]]*) → bool
Verifies passed `grant_type` is part of the client's `grant_types` list.

check_response_type(*response_type: Optional[Union[Literal[token, code, none, id_token], str]]*) → bool
Verifies passed `response_type` is part of the client's `response_types` list.

get_allowed_scope(*scope: str*) → str
Returns the allowed `scope` given the passed `scope`.

> **Note:** Note that the passed `scope` may contain multiple scopes seperated by a space character.

check_scope(*scope: str*) → bool

class AuthorizationCode(*code: str, client_id: str, redirect_uri: str, response_type: str, scope: str, auth_time: int, expires_in: int, code_challenge: Optional[str] = None, code_challenge_method: Optional[Literal[plain, S256]] = None, nonce: Optional[str] = None, user: Optional[Any] = None*)

code: str
Authorization code that the client previously received from the authorization server.

client_id: str
Public identifier for the client. It must also be unique across all clients that the authorization server handles.

redirect_uri: str
After a user successfully authorizes an application, the authorization server will redirect the user back to the application with either an authorization code or access token in the URL. Because the redirect URL will contain sensitive information, it is critical that the service doesn't redirect the user to arbitrary locations.

response_type: str
A string containing the type of the response expected.

scope: str
Scope is a mechanism that limit an application's access to a user's account. An application can request one or more scopes, this information is then presented to the user in the consent screen, and the access token issued to the application will be limited to the scopes granted.

auth_time: int
JSON Web Token Claim indicating the time when the authentication occurred.

expires_in: int
Time delta in which authorization_code will expire.

code_challenge: Optional[str] = None
Only used when RFC 7636, Proof Key for Code Exchange, is used. PKCE works by having the app generate a random value at the beginning of the flow called a Code Verifier. The app hashes the Code Verifier and the result is called the Code Challenge. The app then kicks off the flow in the normal way, except that it includes the Code Challenge in the query string for the request to the Authorization Server.

code_challenge_method: Optional[Literal[plain, S256]] = None
Only used when RFC 7636, Proof Key for Code Exchange, is used. Method used to transform the code verifier into the code challenge.

nonce: Optional[str] = None
Only used when RFC 7636, Proof Key for Code Exchange, is used. Random piece of data.

user: Optional[Any] = None
The user who owns the AuthorizationCode.

check_code_challenge(*code_verifier: str*) → bool

property is_expired
Checks if the authorization_code has expired.

class Token(*access_token: str*, *refresh_token: str*, *scope: str*, *issued_at: int*, *expires_in: int*, *refresh_token_expires_in: int*, *client_id: str*, *token_type: str = 'Bearer'*, *revoked: bool = False*, *user: Optional[Any] = None*)

access_token: str
Token that clients use to make API requests on behalf of the resource owner.

refresh_token: str
Token used by clients to exchange a refresh token for an access token when the access token has expired.

scope: str
Scope is a mechanism that limit an application's access to a user's account. An application can request one or more scopes, this information is then presented to the user in the consent screen, and the access token issued to the application will be limited to the scopes granted.

issued_at: int
Time date in which token was issued at.

expires_in: int
Time delta in which token will expire.

refresh_token_expires_in: int
Time delta in which refresh token will expire.

client_id: str
Public identifier for the client. It must also be unique across all clients that the authorization server handles.

token_type: str = 'Bearer'
Type of token expected.

revoked: bool = False
Flag that indicates whether or not the token has been revoked.

user: Optional[Any] = None
The user who owns the Token.

property is_expired
Checks if the token has expired.

property refresh_token_expired
Checks if refresh token has expired.

4.12 Requests

```
from aioauth import requests
```

Request objects used throughout the project.

---

class Query(*client_id: Optional[str] = None, redirect_uri: str = '', response_type: Optional[str] = None, state: str = '', scope: str = '', nonce: Optional[str] = None, code_challenge_method: Optional[Literal[plain, S256]] = None, code_challenge: Optional[str] = None, response_mode: Optional[Literal[query, form_post, fragment]] = None*)
Object that contains a client's query string portion of a request. Read more on query strings here.

client_id: Optional[str] = None

redirect_uri: str = ''

response_type: Optional[str] = None

state: str = ''

scope: str = ''

nonce: Optional[str] = None

code_challenge_method: Optional[Literal[plain, S256]] = None

code_challenge: Optional[str] = None

response_mode: Optional[Literal[query, form_post, fragment]] = None

class Post(*grant_type: Optional[Literal[authorization_code, password, client_credentials, refresh_token]] = None, client_id: Optional[str] = None, client_secret: Optional[str] = None, redirect_uri: Optional[str] = None, scope: str = '', username: Optional[str] = None, password: Optional[str] = None, refresh_token: Optional[str] = None, code: Optional[str] = None, token: Optional[str] = None, token_type_hint: Optional[str] = None, code_verifier: Optional[str] = None*)
Object that contains a client's post request portion of a request. Read more on post requests here.

**grant_type: Optional[Literal[authorization_code, *password*, client_credentials, *refres*

client_id: Optional[str] = None

client_secret: Optional[str] = None

redirect_uri: Optional[str] = None

scope: str = ''

username: Optional[str] = None

password: Optional[str] = None

refresh_token: Optional[str] = None

code: Optional[str] = None

token: Optional[str] = None

token_type_hint: Optional[str] = None

code_verifier: Optional[str] = None

class BaseRequest(*method: Literal['GET', 'POST'], query: ~TQuery, post: ~TPost, headers: aioauth.collections.HTTPHeaderDict = <factory>, url: str = '', user: Optional[~TUser] = None, settings: aioauth.config.Settings = <factory>*)

method: Literal[GET, POST]

query: TQuery

post: TPost

headers: *aioauth.collections.HTTPHeaderDict*

url: str = ''

user: Optional[TUser] = None

settings: *aioauth.config.Settings*

class Request(*method: Literal[GET, POST], query: aioauth.requests.Query = <factory>, post: aioauth.requests.Post = <factory>, headers: aioauth.collections.HTTPHeaderDict = <factory>, url: str = '', user: Optional[Any] = None, settings: aioauth.config.Settings = <factory>*)

Object that contains a client's complete request.

query: *aioauth.requests.Query*

post: *aioauth.requests.Post*

user: Optional[Any] = None

4.13 Response Type

```
from aioauth import responses
```

Response objects used throughout the project.

---

class ResponseTypeBase(*storage: TStorage*)

Base response type that all other exceptions inherit from.

async validate_request(*request: TRequest*) → *aioauth.models.Client*

class ResponseTypeToken(*storage: TStorage*)

Response type that contains a token.

async create_authorization_response(*request: TRequest, client:* aioauth.models.Client) → *aioauth.responses.TokenResponse*

class ResponseTypeAuthorizationCode(*storage: TStorage*)

Response type that contains an authorization code.

async create_authorization_response(*request: TRequest*, *client:* aioauth.models.Client) → *aioauth.responses.AuthorizationCodeResponse*

class ResponseTypeIdToken(*storage: TStorage*)

async validate_request(*request: TRequest*) → *aioauth.models.Client*

async create_authorization_response(*request: TRequest*, *client:* aioauth.models.Client) → *aioauth.responses.IdTokenResponse*

class ResponseTypeNone(*storage: TStorage*)

async create_authorization_response(*request: TRequest*, *client:* aioauth.models.Client) → *aioauth.responses.NoneResponse*

4.14 Responses

```
from aioauth import responses
```

Response objects used throughout the project.

---

class ErrorResponse(*error: Literal[invalid_request, invalid_client, invalid_grant, invalid_scope, unauthorized_client, unsupported_grant_type, unsupported_response_type, unsupported_token_type, insecure_transport, mismatching_state, method_is_not_allowed, server_error, temporarily_unavailable]*, *description: str*, *error_uri: str = ''*)

Response for errors.

error: Literal[invalid_request, invalid_client, invalid_grant, invalid_scope, unautho

description: str

error_uri: str = ''

class AuthorizationCodeResponse(*code: str*, *scope: str*)

Response for `authorization_code`.

Used by `aioauth.response_type.ResponseTypeAuthorizationCode`.

code: str

scope: str

class NoneResponse

Response for `aioauth.response_type.ResponseTypeNone`.

See: OAuth v2 multiple response types,

class TokenResponse(*expires_in: int*, *refresh_token_expires_in: int*, *access_token: str*, *refresh_token: str*, *scope: str*, *token_type: str = 'Bearer'*)

Response for valid token.

Used by `aioauth.response_type.ResponseTypeToken`.

expires_in: int

refresh_token_expires_in: int

access_token: str

refresh_token: str

scope: str

token_type: str = 'Bearer'

class IdTokenResponse(*id_token: str*)

Response for OpenID id_token.

Used by `aioauth.response_type.ResponseResponseTypeIdTokenTypeToken`.

id_token: str

class TokenActiveIntrospectionResponse(*scope: str, client_id: str, token_type: Literal[access_token, refresh_token], expires_in: int, active: bool = True*)

Response for a valid access token.

Used by `aioauth.server.AuthorizationServer.create_token_introspection_response()`.

scope: str

client_id: str

token_type: Literal[access_token, refresh_token]

expires_in: int

active: bool = True

class TokenInactiveIntrospectionResponse(*active: bool = False*)

For an invalid, revoked or expired token.

Used by `aioauth.server.AuthorizationServer.create_token_introspection_response()`.

active: bool = False

class Response(*content: Dict = <factory>, status_code: http.HTTPStatus = <HTTPStatus.OK: 200>, headers: aioauth.collections.HTTPHeaderDict = <factory>*)

General response class.

Used by `aioauth.server.AuthorizationServer`.

content: Dict

status_code: http.HTTPStatus = 200

headers: *aioauth.collections.HTTPHeaderDict*

4.15 Server

```
from aioauth import server
```

Memory object and interface used to initialize an OAuth2.0 server instance.

> **Warning:** Note that `aioauth.server.AuthorizationServer` is not depedent on any server framework, nor serves at any specific endpoint. Instead, it is used to create an interface that can be used in conjunction with a server framework like `FastAPI` or `aiohttp` to create a fully functional OAuth 2.0 server. Check out the *Examples* portion of the documentation to understand how it can be leveraged in your own project.

---

class AuthorizationServer(*storage: TStorage, response_types: Optional[Dict] = None, grant_types: Optional[Dict] = None*)

Interface for initializing an OAuth 2.0 server.

response_types: Dict[Literal[token, code, none, id_token], Any] = {'code': aioauth.r

grant_types: Dict[Literal[authorization_code, password, client_credentials, refresh_t

is_secure_transport(*request: TRequest*) → bool

Verifies the request was sent via a protected SSL tunnel.

> **Note:** This method simply checks if the request URL contains `https://` at the start of it. It does **not** ensure if the SSL certificate is valid.

Parameters **request** – *aioauth.requests.Request* object.

Returns Flag representing whether or not the transport is secure.

validate_request(*request: TRequest, allowed_methods: List[Literal[GET, POST]]*)

async create_token_introspection_response(*request: TRequest*) → *aioauth.responses.Response*

Returns a response object with introspection of the passed token. For more information see RFC7662 section 2.1.

> **Note:** The API endpoint that leverages this function is usually `/introspect`.

Example

Below is an example utilizing FastAPI as the server framework.

```
from aioauth_fastapi.utils import to_oauth2_request, to_fastapi_response

@app.get("/token/introspect")
async def introspect(request: fastapi.Request) -> fastapi.Response:
    # Converts a fastapi.Request to an aioauth.Request.
    oauth2_request: aioauth.Request = await to_oauth2_request(request)
    # Creates the response via this function call.
    oauth2_response: aioauth.Response = await server.create_token_
↪introspection_response(oauth2_request)
    # Converts an aioauth.Response to a fastapi.Response.
    response: fastapi.Response = await to_fastapi_response(oauth2_response)
    return response
```

Parameters **request** – An *aioauth.requests.Request* object.

Returns An *aioauth.responses.Response* object.

Return type response

get_client_credentials(*request: TRequest*) → Tuple[str, str]

async create_token_response(*request: TRequest*) → *aioauth.responses.Response*

Endpoint to obtain an access and/or ID token by presenting an authorization grant or refresh token. Validates a token request and creates a token response. For more information see RFC6749 section 4.1.3.

Note: The API endpoint that leverages this function is usually `/token`.

Example

Below is an example utilizing FastAPI as the server framework.

```
from aioauth_fastapi.utils import to_oauth2_request, to_fastapi_response

@app.post("/token")
async def token(request: fastapi.Request) -> fastapi.Response:
    # Converts a fastapi.Request to an aioauth.Request.
    oauth2_request: aioauth.Request = await to_oauth2_request(request)
    # Creates the response via this function call.
    oauth2_response: aioauth.Response = await server.create_token_
↪response(oauth2_request)
    # Converts an aioauth.Response to a fastapi.Response.
    response: fastapi.Response = await to_fastapi_response(oauth2_response)
    return response
```

Parameters **request** – An *aioauth.requests.Request* object.

Returns An *aioauth.responses.Response* object.

Return type response

async create_authorization_response(*request: TRequest*) → *aioauth.responses.Response*

Endpoint to interact with the resource owner and obtain an authorization grant. Validate authorization request and create authorization response. For more information see RFC6749 section 4.1.1.

Note: The API endpoint that leverages this function is usually `/authorize`.

Example

Below is an example utilizing FastAPI as the server framework.

```
from aioauth.fastapi.utils import to_oauth2_request, to_fastapi_response

@app.post("/authorize")
async def authorize(request: fastapi.Request) -> fastapi.Response:
    # Converts a fastapi.Request to an aioauth.Request.
    oauth2_request: aioauth.Request = await to_oauth2_request(request)
    # Creates the response via this function call.
    oauth2_response: aioauth.Response = await server.create_authorization_
↪response(oauth2_request)
    # Converts an aioauth.Response to a fastapi.Response.
    response: fastapi.Response = await to_fastapi_response(oauth2_response)
    return response
```

Parameters **request** – An *aioauth.requests.Request* object.

Returns An *aioauth.responses.Response* object.

Return type response

async revoke_token(*request: TRequest*) → *aioauth.responses.Response*

Endpoint to revoke an access token or refresh token. For more information see RFC7009.

Note: The API endpoint that leverages this function is usually `/revoke`.

Example

Below is an example utilizing FastAPI as the server framework.

```
from aioauth_fastapi.utils import to_oauth2_request, to_fastapi_response

@app.post("/revoke")
async def revoke(request: fastapi.Request) -> fastapi.Response:
    # Converts a fastapi.Request to an aioauth.Request.
    oauth2_request: aioauth.Request = await to_oauth2_request(request)
    # Creates the response via this function call.
    oauth2_response: aioauth.Response = await server.revoke_token(oauth2_
↪request)
    # Converts an aioauth.Response to a fastapi.Response.
    response: fastapi.Response = await to_fastapi_response(oauth2_response)
    return response
```

Parameters **request** – An *aioauth.requests.Request* object.

Returns An *aioauth.responses.Response* object.

Return type response

4.16 Storage

```
from aioauth import storage
```

Storage helper class for storing and retrieving client and resource owner information. See the examples on the sidebar to view this in action.

class BaseStorage

async create_token(*request: TRequest*, *client_id: str*, *scope: str*, *access_token: str*, *refresh_token: str*) → TToken

Generates a user token and stores it in the database.

Warning: Generated token *must* be stored in the database.

Note: Method is used by all core grant types, but only used for `aioauth.response_type.ResponseTypeToken`.

Parameters

- **request** – An `aioauth.requests.Request`.
- **client_id** – A user client ID.
- **scope** – The scopes for the token.

Returns The new generated `aioauth.models.Token`.

async get_token(*request: TRequest, client_id: str, token_type: Optional[Literal[access_token, refresh_token]] = 'refresh_token', access_token: Optional[str] = None, refresh_token: Optional[str] = None*) → Optional[TToken]

Gets existing token from the database.

Note: Method is used by `aioauth.server.AuthorizationServer`, and by the grant type `aioauth.grant_types.RefreshTokenGrantType`.

Parameters

- **request** – An `aioauth.requests.Request`.
- **client_id** – A user client ID.
- **access_token** – The user access token.
- **refresh_token** – The user refresh token.

Returns An optional `aioauth.models.Token` object.

async create_authorization_code(*request: TRequest, client_id: str, scope: str, response_type: Literal[token, code, none, id_token], redirect_uri: str, code_challenge_method: Optional[Literal[plain, S256]], code_challenge: Optional[str], code: str, **kwargs*) → TAuthorizationCode

Generates an authorization token and stores it in the database.

Warning: Generated authorization token *must* be stored in the database.

Note: This must is used by the response type `aioauth.respose_type.ResponseTypeAuthorizationCode`.

Parameters

- **request** – An `aioauth.requests.Request`.
- **client_id** – A user client ID.
- **scope** – The scopes for the token.
- **response_type** – An `aioauth.types.ResponseType`.

- **redirect_uri** – The redirect URI.
- **code_challenge_method** – An `aioauth.types.CodeChallengeMethod`.
- **code_challenge** – Code challenge string.

Returns An *aioauth.models.AuthorizationCode* object.

async get_id_token(*request: TRequest*, *client_id: str*, *scope: str*, *response_type: Literal[token, code, none, id_token]*, *redirect_uri: str*, ***kwargs*) → str

Returns an id_token. For more information see OpenID Connect Core 1.0 incorporating errata set 1 section 2.

Note: Method is used by response type *aioauth.response_type.ResponseTypeIdToken* and `aioauth.oidc.core.grant_type.AuthorizationCodeGrantType`.

async get_client(*request: TRequest*, *client_id: str*, *client_secret: Optional[str] = None*) → Optional[TClient]

Gets existing client from the database if it exists.

Warning: If client does not exists in database this method *must* return `None` to indicate to the validator that the requested `client_id` does not exist or is invalid.

Note: This method is used by all core grant types, as well as all core response types.

Parameters

- **request** – An *aioauth.requests.Request*.
- **client_id** – A user client ID.
- **client_secret** – An optional user client secret.

Returns An optional *aioauth.models.Client* object.

async authenticate(*request: TRequest*) → bool

Authenticates a user.

Note: This method is used by the grant type *aioauth.grant_type.PasswordGrantType*.

Parameters **request** – An *aioauth.requests.Request*.

Returns Boolean indicating whether or not the user was authenticated successfully.

async get_authorization_code(*request: TRequest*, *client_id: str*, *code: str*) → Optional[TAuthorizationCode]

Gets existing authorization code from the database if it exists.

Warning: If authorization code does not exists this function *must* return `None` to indicate to the validator that the requested authorization code does not exist or is invalid.

Note: This method is used by the grant type `aioauth.grant_type.AuthorizationCodeGrantType`.

Parameters

- **request** – An `aioauth.requests.Request`.
- **client_id** – A user client ID.
- **code** – An authorization code.

Returns An optional `aioauth.models.AuthorizationCode`.

async delete_authorization_code(*request: TRequest*, *client_id: str*, *code: str*) → None

Deletes authorization code from database.

Note: This method is used by the grant type `aioauth.grant_type.AuthorizationCodeGrantType`.

Parameters

- **request** – An `aioauth.requests.Request`.
- **client_id** – A user client ID.
- **code** – An authorization code.

async revoke_token(*request: TRequest*, *token_type: Optional[Literal[access_token, refresh_token]] = 'refresh_token'*, *access_token: Optional[str] = None*, *refresh_token: Optional[str] = None*) → None

Revokes a token's from the database.

Note: This method *must* set `revoked` to `True` for an existing token record. This method is used by the grant type `aioauth.grant_types.RefreshTokenGrantType`.

Parameters

- **request** – An `aioauth.requests.Request`.
- **refresh_token** – The user refresh token.

4.17 Types

```
from aioauth import types
```

Containers that contain constants used throughout the project.

4.18 Utils

```
from aioauth import utils
```

Contains helper functions that is used throughout the project that doesn't pertain to a specific file or module.

---

get_authorization_scheme_param(*authorization_header_value: str*) → Tuple[str, str]

Retrieves the authorization schema parameters from the authorization header.

Parameters **authorization_header_value** – Value of the authorization header.

Returns Tuple of the format `(scheme, param)`.

enforce_str(*scope: List*) → str

Converts a list of scopes to a space separated string.

Note: If a string is passed to this method it will simply return an empty string back. Use *enforce_list()* to convert strings to scope lists.

Parameters **scope** – An iterable or string that contains a list of scope.

Returns A string of scopes seperated by spaces.

Raises **TypeError** – The `scope` value passed is not of the proper type.

enforce_list(*scope: Optional[Union[str, List, Set, Tuple]]*) → List

Converts a space separated string to a list of scopes.

Note: If an iterable is passed to this method it will return a list representation of the iterable. Use *enforce_str()* to convert iterables to a scope string.

Parameters **scope** – An iterable or string that contains scopes.

Returns A list of scopes.

generate_token(*length: int = 30, chars: str = 'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789'*) → str

Generates a non-guessable OAuth token. OAuth (1 and 2) does not specify the format of tokens except that they should be strings of random characters. Tokens should not be guessable and entropy when generating the random characters is important. Which is why SystemRandom is used instead of the default random.choice method.

Parameters

- **length** – Length of the generated token.
- **chars** – The characters to use to generate the string.

Returns Random string of length `length` and characters in `chars`.

build_uri(*url: str, query_params: Optional[Dict] = None, fragment: Optional[Dict] = None*) → str

Builds an URI string from passed `url`, `query_params`, and `fragment`.

Parameters

- **url** – URL string.

- **query_params** – Paramaters that contain the query.
- **fragment** – Fragment of the page.

Returns URL containing the original `url`, and the added `query_params` and `fragment`.

encode_auth_headers(*client_id: str*, *client_secret: str*) → *aioauth.collections.HTTPHeaderDict*

Encodes the authentication header using base64 encoding.

Parameters

- **client_id** – The client's id.
- **client_secret** – The client's secret.

Returns A case insensitive dictionary that contains the `Authorization` header set to `basic` and the authorization header.

decode_auth_headers(*authorization: str*) → Tuple[str, str]

Decodes an encoded HTTP basic authentication string. Returns a tuple of the form `(client_id, client_secret)`, and raises a *`aioauth.errors.InvalidClientError`* exception if nothing could be decoded.

Parameters **authorization** – Authorization header string.

Returns Tuple of the form `(client_id, client_secret)`.

Raises **ValueError** – Invalid *authorization* header string.

create_s256_code_challenge(*code_verifier: str*) → str

Create S256 code challenge with the passed `code_verifier`.

Note: This function implements `base64url(sha256(ascii(code_verifier)))`.

Parameters **code_verifier** – Code verifier string.

Returns Representation of the S256 code challenge with the passed `code_verifier`.

catch_errors_and_unavailability(*skip_redirect_on_exc: Tuple[Type[aioauth.errors.OAuth2Error], ...] = (<class 'aioauth.errors.OAuth2Error'>,)*) → Callable[[...], Callable[[...], Coroutine[Any, Any, *aioauth.responses.Response*]]]

Decorator that adds error catching to the function passed.

Parameters **f** – A callable.

Returns A callable with error catching capabilities.

CHAPTER

FIVE

INDICES AND TABLES

- genindex
- modindex
- search

PYTHON MODULE INDEX

a

aioauth.collections, 11
aioauth.config, 11
aioauth.constances, 12
aioauth.errors, 12
aioauth.grant_type, 14
aioauth.models, 15
aioauth.requests, 18
aioauth.response_type, 19
aioauth.responses, 20
aioauth.server, 21
aioauth.storage, 24
aioauth.types, 27
aioauth.utils, 28

INDEX

A

access_token (*Token attribute*), 17
access_token (*TokenResponse attribute*), 20
active (*TokenActiveIntrospectionResponse attribute*), 21
active (*TokenInactiveIntrospectionResponse attribute*), 21
aioauth.collections
    module, 11
aioauth.config
    module, 11
aioauth.constances
    module, 12
aioauth.errors
    module, 12
aioauth.grant_type
    module, 14
aioauth.models
    module, 15
aioauth.requests
    module, 18
aioauth.response_type
    module, 19
aioauth.responses
    module, 20
aioauth.server
    module, 21
aioauth.storage
    module, 24
aioauth.types
    module, 27
aioauth.utils
    module, 28
auth_time (*AuthorizationCode attribute*), 17
authenticate() (*BaseStorage method*), 26
AUTHORIZATION_CODE_EXPIRES_IN (*Settings attribute*), 12
AuthorizationCode (*class in aioauth.models*), 16
AuthorizationCodeGrantType (*class in aioauth.grant_type*), 14
AuthorizationCodeResponse (*class in aioauth.responses*), 20
AuthorizationServer (*class in aioauth.server*), 22
AVAILABLE (*Settings attribute*), 12

B

BaseRequest (*class in aioauth.requests*), 19
BaseStorage (*class in aioauth.storage*), 24
build_uri() (*in module aioauth.utils*), 28

C

catch_errors_and_unavailability() (*in module aioauth.utils*), 29
check_code_challenge() (*AuthorizationCode method*), 17
check_grant_type() (*Client method*), 16
check_redirect_uri() (*Client method*), 16
check_response_type() (*Client method*), 16
check_scope() (*Client method*), 16
Client (*class in aioauth.models*), 15
client_id (*AuthorizationCode attribute*), 16
client_id (*Client attribute*), 15
client_id (*Post attribute*), 18
client_id (*Query attribute*), 18
client_id (*Token attribute*), 17
client_id (*TokenActiveIntrospectionResponse attribute*), 21
client_secret (*Client attribute*), 15
client_secret (*Post attribute*), 18
ClientCredentialsGrantType (*class in aioauth.grant_type*), 15
code (*AuthorizationCode attribute*), 16
code (*AuthorizationCodeResponse attribute*), 20
code (*Post attribute*), 19
code_challenge (*AuthorizationCode attribute*), 17
code_challenge (*Query attribute*), 18
code_challenge_method (*AuthorizationCode attribute*), 17
code_challenge_method (*Query attribute*), 18
code_verifier (*Post attribute*), 19
content (*Response attribute*), 21
create_authorization_code() (*BaseStorage method*), 25

create_authorization_response() (*AuthorizationServer method*), 23
create_authorization_response() (*ResponseTypeAuthorizationCode method*), 19
create_authorization_response() (*ResponseTypeIdToken method*), 20
create_authorization_response() (*ResponseTypeNone method*), 20
create_authorization_response() (*ResponseTypeToken method*), 19
create_s256_code_challenge() (*in module aioauth.utils*), 29
create_token() (*BaseStorage method*), 24
create_token_introspection_response() (*AuthorizationServer method*), 22
create_token_response() (*AuthorizationCodeGrantType method*), 15
create_token_response() (*AuthorizationServer method*), 22
create_token_response() (*GrantTypeBase method*), 14
create_token_response() (*RefreshTokenGrantType method*), 15

D

decode_auth_headers() (*in module aioauth.utils*), 29
default_headers (*in module aioauth.constances*), 12
delete_authorization_code() (*BaseStorage method*), 27
description (*ErrorResponse attribute*), 20
description (*InsecureTransportError attribute*), 13
description (*MethodNotAllowedError attribute*), 12
description (*MismatchingStateError attribute*), 13

E

encode_auth_headers() (*in module aioauth.utils*), 29
enforce_list() (*in module aioauth.utils*), 28
enforce_str() (*in module aioauth.utils*), 28
error (*ErrorResponse attribute*), 20
error (*InsecureTransportError attribute*), 13
error (*InvalidClientError attribute*), 13
error (*InvalidGrantError attribute*), 13
error (*InvalidRedirectURIError attribute*), 14
error (*InvalidRequestError attribute*), 12
error (*InvalidScopeError attribute*), 14
error (*MethodNotAllowedError attribute*), 12
error (*MismatchingStateError attribute*), 13
error (*ServerError attribute*), 14
error (*TemporarilyUnavailableError attribute*), 14
error (*UnauthorizedClientError attribute*), 13
error (*UnsupportedGrantTypeError attribute*), 13
error (*UnsupportedResponseTypeError attribute*), 13
error (*UnsupportedTokenTypeError attribute*), 14
error_uri (*ErrorResponse attribute*), 20
ERROR_URI (*Settings attribute*), 12
ErrorResponse (*class in aioauth.responses*), 20
expires_in (*AuthorizationCode attribute*), 17
expires_in (*Token attribute*), 17
expires_in (*TokenActiveIntrospectionResponse attribute*), 21
expires_in (*TokenResponse attribute*), 20

G

generate_token() (*in module aioauth.utils*), 28
get_allowed_scope() (*Client method*), 16
get_authorization_code() (*BaseStorage method*), 26
get_authorization_scheme_param() (*in module aioauth.utils*), 28
get_client() (*BaseStorage method*), 26
get_client_credentials() (*AuthorizationServer method*), 22
get_id_token() (*BaseStorage method*), 26
get_token() (*BaseStorage method*), 25
grant_type (*Post attribute*), 18
grant_types (*AuthorizationServer attribute*), 22
grant_types (*Client attribute*), 16
GrantTypeBase (*class in aioauth.grant_type*), 14

H

headers (*BaseRequest attribute*), 19
headers (*Response attribute*), 21
HTTPHeaderDict (*class in aioauth.collections*), 11

I

id_token (*IdTokenResponse attribute*), 21
IdTokenResponse (*class in aioauth.responses*), 21
INSECURE_TRANSPORT (*Settings attribute*), 12
InsecureTransportError, 13
InvalidClientError, 12
InvalidGrantError, 13
InvalidRedirectURIError, 14
InvalidRequestError, 12
InvalidScopeError, 13
is_expired() (*AuthorizationCode property*), 17
is_expired() (*Token property*), 18
is_secure_transport() (*AuthorizationServer method*), 22
issued_at (*Token attribute*), 17

M

method (*BaseRequest attribute*), 19
MethodNotAllowedError, 12
MismatchingStateError, 13

module
    aioauth.collections, 11
    aioauth.config, 11
    aioauth.constances, 12
    aioauth.errors, 12
    aioauth.grant_type, 14
    aioauth.models, 15
    aioauth.requests, 18
    aioauth.response_type, 19
    aioauth.responses, 20
    aioauth.server, 21
    aioauth.storage, 24
    aioauth.types, 27
    aioauth.utils, 28

N

nonce (*AuthorizationCode attribute*), 17
nonce (*Query attribute*), 18
NoneResponse (*class in aioauth.responses*), 20

P

password (*Post attribute*), 18
PasswordGrantType (*class in aioauth.grant_type*), 15
post (*BaseRequest attribute*), 19
Post (*class in aioauth.requests*), 18
post (*Request attribute*), 19

Q

query (*BaseRequest attribute*), 19
Query (*class in aioauth.requests*), 18
query (*Request attribute*), 19

R

redirect_uri (*AuthorizationCode attribute*), 16
redirect_uri (*Post attribute*), 18
redirect_uri (*Query attribute*), 18
redirect_uris (*Client attribute*), 16
refresh_token (*Post attribute*), 19
refresh_token (*Token attribute*), 17
refresh_token (*TokenResponse attribute*), 20
refresh_token_expired() (*Token property*), 18
REFRESH_TOKEN_EXPIRES_IN (*Settings attribute*), 11
refresh_token_expires_in (*Token attribute*), 17
refresh_token_expires_in (*TokenResponse attribute*), 20
RefreshTokenGrantType (*class in aioauth.grant_type*), 15
Request (*class in aioauth.requests*), 19
Response (*class in aioauth.responses*), 21
response_mode (*Query attribute*), 18
response_type (*AuthorizationCode attribute*), 16
response_type (*Query attribute*), 18
response_types (*AuthorizationServer attribute*), 22
response_types (*Client attribute*), 16
ResponseTypeAuthorizationCode (*class in aioauth.response_type*), 19
ResponseTypeBase (*class in aioauth.response_type*), 19
ResponseTypeIdToken (*class in aioauth.response_type*), 20
ResponseTypeNone (*class in aioauth.response_type*), 20
ResponseTypeToken (*class in aioauth.response_type*), 19
revoke_token() (*AuthorizationServer method*), 24
revoke_token() (*BaseStorage method*), 27
revoked (*Token attribute*), 17

S

scope (*AuthorizationCode attribute*), 16
scope (*AuthorizationCodeResponse attribute*), 20
scope (*Client attribute*), 16
scope (*Post attribute*), 18
scope (*Query attribute*), 18
scope (*Token attribute*), 17
scope (*TokenActiveIntrospectionResponse attribute*), 21
scope (*TokenResponse attribute*), 21
ServerError, 14
settings (*BaseRequest attribute*), 19
Settings (*class in aioauth.config*), 11
state (*Query attribute*), 18
status_code (*InvalidClientError attribute*), 13
status_code (*MethodNotAllowedError attribute*), 12
status_code (*Response attribute*), 21

T

TemporarilyUnavailableError, 14
Token (*class in aioauth.models*), 17
token (*Post attribute*), 19
TOKEN_EXPIRES_IN (*Settings attribute*), 11
token_type (*Token attribute*), 17
token_type (*TokenActiveIntrospectionResponse attribute*), 21
token_type (*TokenResponse attribute*), 21
token_type_hint (*Post attribute*), 19
TokenActiveIntrospectionResponse (*class in aioauth.responses*), 21
TokenInactiveIntrospectionResponse (*class in aioauth.responses*), 21
TokenResponse (*class in aioauth.responses*), 20

U

UnauthorizedClientError, 13
UnsupportedGrantTypeError, 13
UnsupportedResponseTypeError, 13
UnsupportedTokenTypeError, 14

`url` (*BaseRequest attribute*), 19
`user` (*AuthorizationCode attribute*), 17
`user` (*BaseRequest attribute*), 19
`user` (*Client attribute*), 16
`user` (*Request attribute*), 19
`user` (*Token attribute*), 18
`username` (*Post attribute*), 18

V

`validate_request()` (*AuthorizationCodeGrantType method*), 15
`validate_request()` (*AuthorizationServer method*), 22
`validate_request()` (*ClientCredentialsGrantType method*), 15
`validate_request()` (*GrantTypeBase method*), 14
`validate_request()` (*PasswordGrantType method*), 15
`validate_request()` (*RefreshTokenGrantType method*), 15
`validate_request()` (*ResponseTypeBase method*), 19
`validate_request()` (*ResponseTypeIdToken method*), 20